

FAST – NUCES



Object Oriented Programming Lab Manual

Course Teacher	Ms. Hina Iqbal
Lab Instructor	Mr. Ahmad Jawad Mustasim
Lab TA	Ms. Mania Shakeel
Section	BSE-2B
Semester	Spring 2026

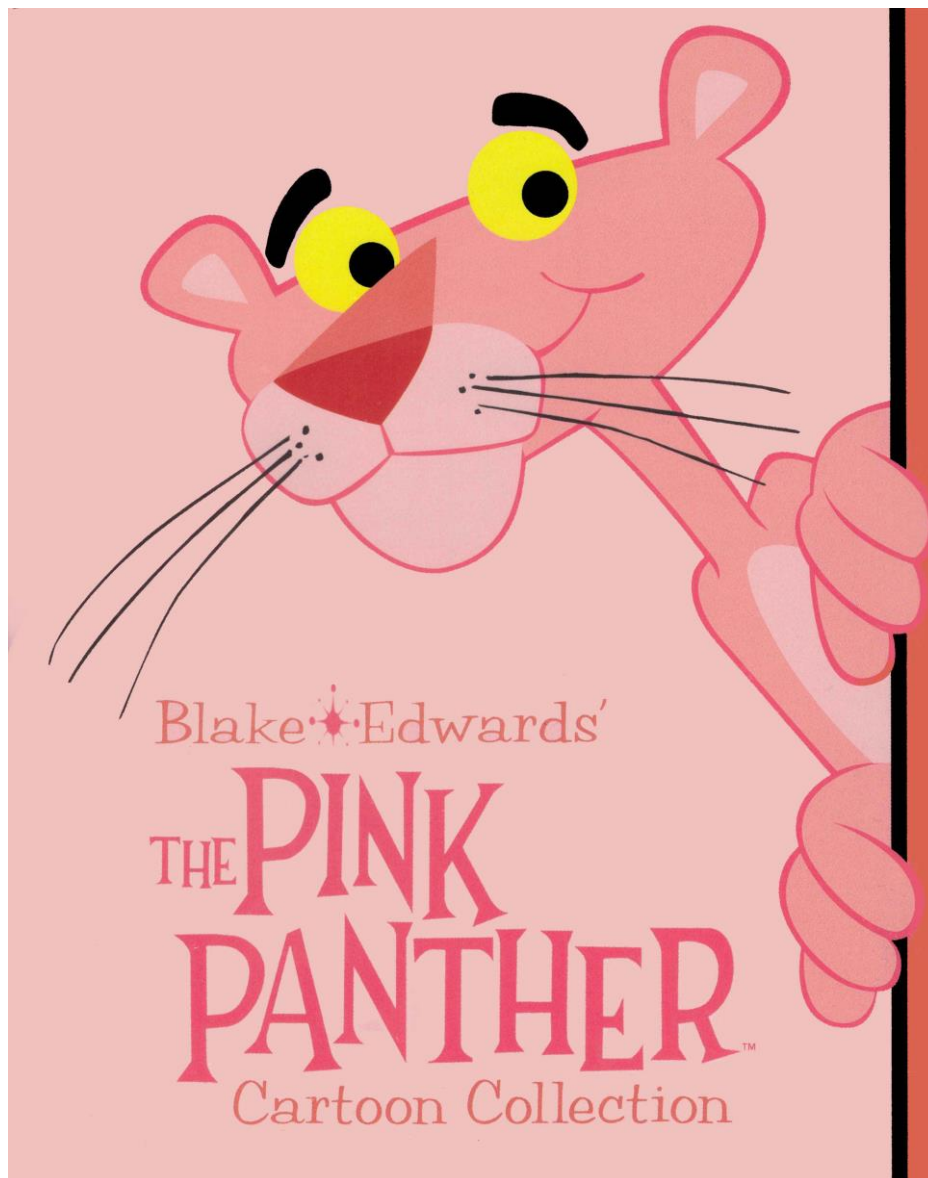
Department of Computer Science

FAST-NU, Lahore, Pakistan

Lab 9 – Operator Overloading, friend function/ classes

Objectives

- Unary and Binary operator overloading
 - Friend functions, Stream operators (<<, >>)
 - Comparison and Assignment operators
 - Advanced concepts (friend class, mixed operands)
-



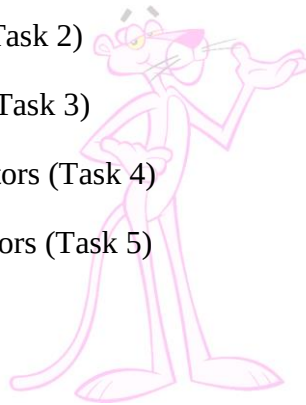
File Organization Instructions

File 1: Task1.cpp

- Implement **Task 1 only**
- Use **member function operator overloading**
- Do NOT modify this file later
This file serves as the **base version**

File 2: Task2_Onwards.cpp

- Copy code from **Task 1**
- Modify it for:
 - Friend functions (Task 2)
 - Stream operators (Task 3)
 - Comparison operators (Task 4)
 - Assignment operators (Task 5)
 - All bonus task



Read carefully and attempt following task

Task 1: Panther Operator Core (Unary & Binary Overloading)

Class Name: SpyAgent

Private Members:

- string name
- int missions

Public Members:

- Constructors
- void display()

Operator Overloading (Member Functions)**Unary Operators:**

- ++ (Pre-increment) → Promote agent (increase missions)
- -- (Pre-decrement) → Demote agent (decrease missions)

Binary Operators:

- + → Combine missions
- - → Mission difference
- * → Multiply missions
- / → Divide missions

**Sample Output**

Agent Panther promoted. Missions: 11
Agent Clouseau demoted. Missions: 4

Combined Missions: 15
Mission Difference: 7
Multiplied Missions: 44
Division: 2

Task 2: Binary and Unary Operators using Friend Functions

Copy **Task 1** and modify it.

Requirement:

Convert all task1 operators into **friend functions**:

Instructions:

- Define friend functions outside the class
- Access private data using friend keyword

Task 3: Friend Function + Stream Overloading (Spy Logger Access)

Class Name: SpyLog

Private Members:

- string agentName
- string operation
- int clearance



Friend Function:

- friend void printLog(const SpyLog& log);

Operator Overloading:

- >> → Input
- << → Output

Sample Output

Enter agent name: Panther

Enter operation: Vault Hack

Enter clearance level: 5

Pink Panther is processing log...

Agent: Panther | Operation: Vault Hack | Clearance: 5

Task 4: Comparison Operator Overloading

Extend SpyAgent class

Overload:

- `==` → Equal missions
- `!=` → Not equal
- `>` → More missions

Expected Behavior

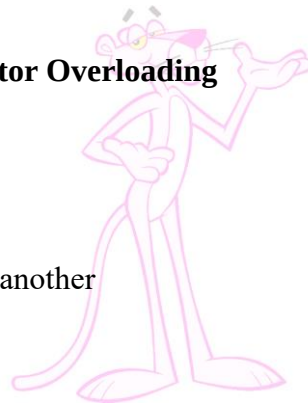
Agent Panther has more missions than Clouseau
Agents are not equal

Task 5: Assignment Operator Overloading

Extend SpyAgent

Overload:

- `=` → Assign one object to another
- `+=` → Add missions
- `-=` → Subtract missions



Requirements:

- Handle self-assignment
- Maintain correct data copying

Example

Agent Panther assigned data from Clouseau
Agent Panther updated missions: 20

Task 6: Additional Comparison Operators

Overload:

- $< \rightarrow$ Less missions
- $<= \rightarrow$ Less or equal
- $>= \rightarrow$ Greater or equal

Goal:

- Enable full comparison between objects
- Useful for sorting and ranking agents

Task 7: Mixed Operand Overloading

Work with object + primitive types

Implement:

Case 1:

- `SpyAgent + int`

Case 2:

- `int + SpyAgent` (**must use friend function**)

Case 3:

- `SpyAgent - int`

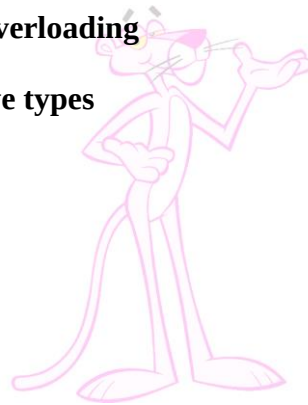
Example Usage

```
SpyAgent a("Panther", 10);
```

```
a + 5
```

```
5 + a
```

```
a - 3
```



Hint:

- Member → handles obj + value
- Friend → handles value + obj

Task 8: Friend Class (SpyHQ Access System)

Create a new class: SpyHQ

Requirements:

- Make SpyHQ a **friend class** of SpyAgent
- Access and display private data of agents

Goal:

- Understand full class-level access using friend class

Task 9: Forward Class Declaration

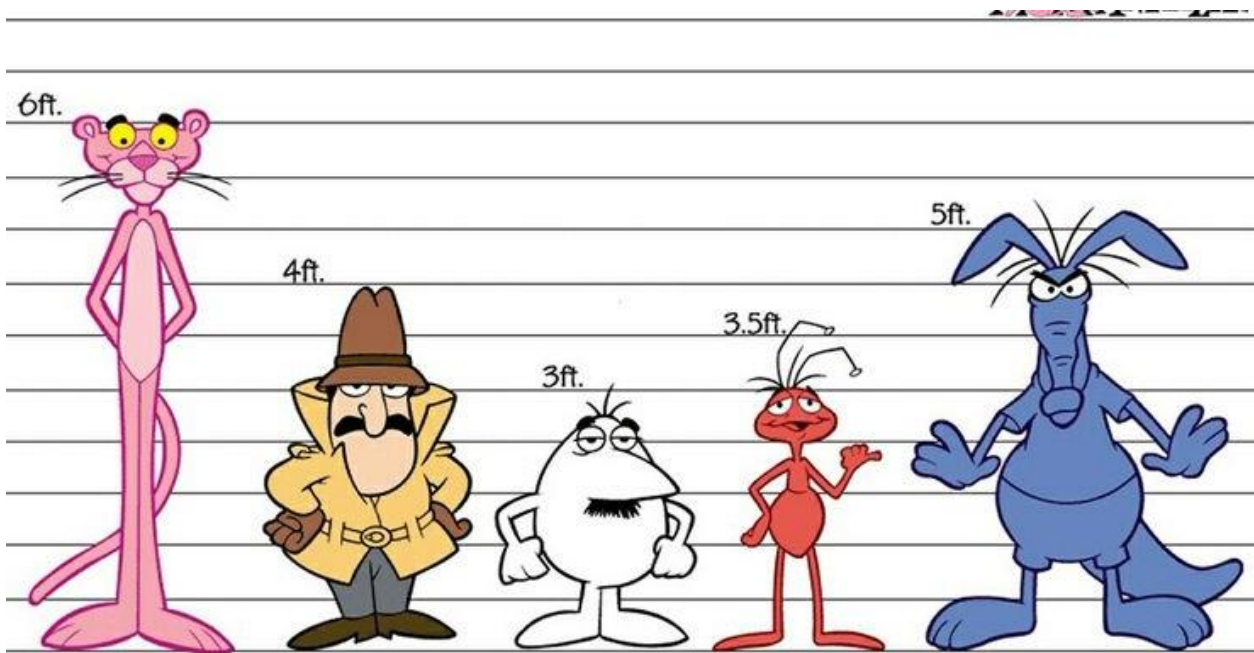
Use **forward declaration** where needed

(A forward class declaration is a way to tell the compiler that a class exists before its full definition, so it can be referenced in other class declarations.)

Goal:

- Avoid circular dependency between classes
- Practice proper class structuring

-----Happy Coding! 😊-----



Submission Instructions

- Submit your tasks on Google Classroom within given deadline
- Also submit screenshots of each task for example “LAB01-TASK01.png”, “LAB01-TASK02.png” and so on
- Submit 2 cpp files for this task: “task1.cpp” and “task2_onwards.cpp”
- “.cpp” is the actual file where all your code is saved. Always submit .cpp files when asked for submission

